

Model Context Protocol (MCP): The Standard That Makes AI Actually Useful

Published March 22, 2026

Modern AI assistants are impressive in isolation, but their real value comes from connecting to the systems where your work actually lives: your codebase, your database, your Slack, your docs. The problem is that every such connection has historically been a one-off engineering project. MCP fixes that. This blog explores the architecture, protocol design, and practical implementation of the Model Context Protocol.

1. The Problem: Context Is Scattered

When an AI generates a response, it can only work with what it can 'see' -- its context. Context can be as simple as a chat history, or as complex as a distributed system spread across multiple services. In a real-world engineering environment, context is scattered across:

- Jira -- feature descriptions and ticket details
- Databases -- schemas, data, and migration history
- Slack -- team communication and decisions
- Google Drive -- guidelines, design docs, and specs
- GitHub -- code, pull requests, and CI/CD pipelines

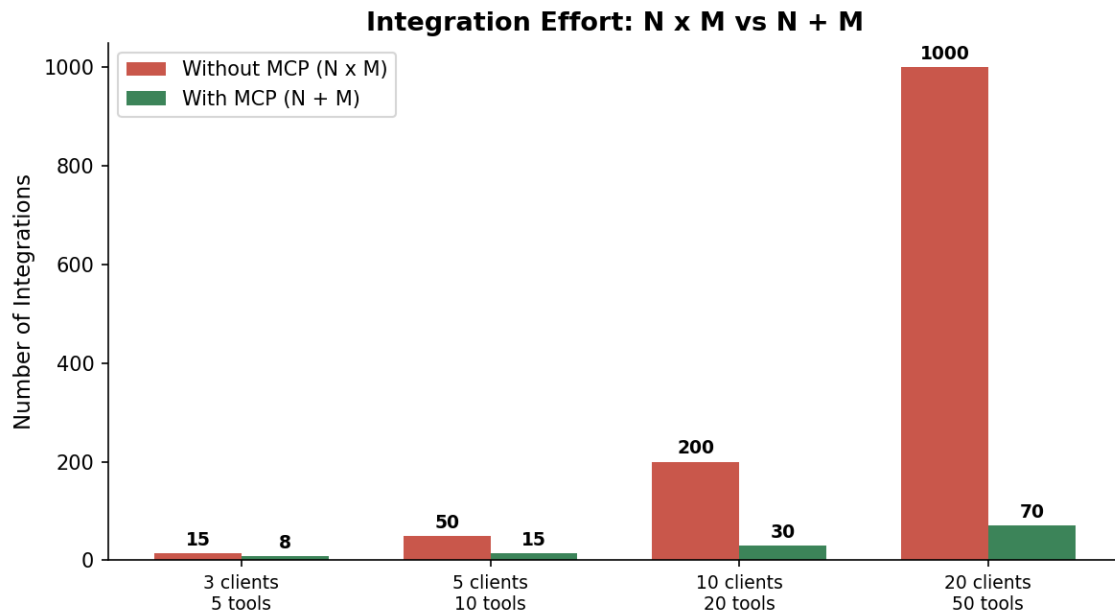
All of that scattered context must be assembled and fed into the model before it can do anything useful. Context assembly is a great toil. Historically, every team built their own bespoke glue code to pull this off -- custom scripts, API wrappers, and fragile integrations that break at the worst possible time.

2. The Integration Tax: $N \times M$

If you have N AI chatbots (ChatGPT, Claude, Gemini, Perplexity, etc.) and M tools (GitHub, Slack, Jira, databases, etc.), you're on the hook for $N \times M$ custom integrations. Each one requires:

- Custom API clients for each service
- Authentication and authorization logic
- Error handling and retry mechanisms
- Ongoing maintenance, security reviews, and version updates

This does not scale. With 5 AI clients and 10 tools, you need 50 integrations. With 20 clients and 50 tools, that's 1,000. The operational burden grows multiplicatively, and security fragmentation becomes a serious risk.



3. What Is MCP?

"Giving LLMs the power of calling functions."

MCP -- the Model Context Protocol -- is an open standard that defines a common protocol for AI clients (hosts) to connect to external tools and data sources. Instead of every AI tool inventing its own integration format, MCP provides a shared language.

The math changes dramatically. Instead of $N \times M$ custom integrations, you build N MCP clients and M MCP servers -- a total of $N + M$ components. Add a new MCP Server for a service, and every MCP-compatible client can use it immediately. Add a new MCP Client, and it instantly gains access to every existing server. Zero additional integration work on either side.

4. The Business Case: MCP Ecosystem

The MCP ecosystem creates a powerful network effect. On the supply side, service providers (Salesforce, GitHub, Slack, Google Drive) build MCP Servers -- each wraps a tool or service behind a standard interface. On the demand side, AI chatbots (ChatGPT, Claude, Gemini) become MCP Client compliant -- they can connect to any MCP Server out of the box.

The result: $A \times B$ integrations with zero custom code per pair. Every new server is instantly usable by every client. Every new client gains access to every server. The ecosystem grows multiplicatively with no extra integration work. Becoming AI-native via MCPs is a strategic advantage that scales.

Side	Role	Examples
MCP Servers (A)	Wrap services behind MCP	Salesforce, GitHub, Slack
MCP Clients (B)	AI apps that consume MCP	ChatGPT, Claude, Gemini
Integrations	A x B with zero glue code	Every pair works instantly

5. Architecture: Host, LLM, and MCP Server

MCP introduces three distinct components that work together in a layered architecture:

Component	Role	Examples
Host	AI app the user interacts with	Claude Code, GitHub Copilot, LangChain
LLM	Model that reasons & generates	Gemini, GPT-4, Claude
MCP Server	Backend wrapped in MCP protocol	Google Drive, GitHub, Slack

The Host contains both an LLM and an MCP Client -- the component responsible for discovering and calling MCP Servers on the LLM's behalf. The LLM never calls tools directly; it tells the Host what it needs, and the Host's MCP Client does the actual work.

Request Dataflow: 8-Step Process

When a user asks something like 'summarize the latest PR and check if the ticket is closed', the request flows through all three components:

Step	Direction	Action
1	User -> Host	User sends prompt
2	Host -> LLM	Host forwards prompt to LLM
3	LLM -> Host	LLM requests tool call (not a final answer)
4	Host -> MCP Server	MCP Client calls the appropriate server
5	MCP Server -> Host	Server returns tool result
6	Host -> LLM	Host sends tool result as additional context
7	LLM -> Host	LLM generates final response
8	Host -> User	Host delivers response to user

6. The 1:1 Connection Model

Each MCP Client maintains a dedicated 1:1 connection per server. The client manages multiple such connections, but each channel is exclusive to one server -- servers are never shared across connections. Think of it like a mobile phone with multiple SIM cards: each SIM connects to exactly one provider, and they don't share state.

This design delivers two key benefits:

- Decoupling -- servers are isolated from each other. Adding, removing, or updating one server has no effect on the others.
- Parallelism -- multiple tool calls to different servers can execute concurrently without contention or shared state.

7. MCP Primitives

Every MCP Server exposes its capabilities through three types of primitives:

7.1 Tools (Dynamic)

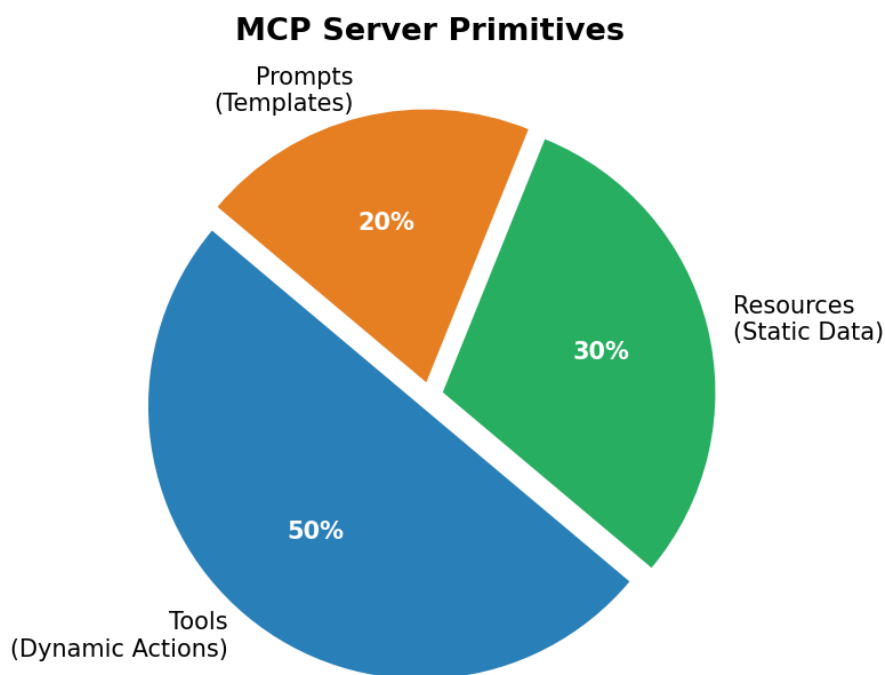
Actions the AI can invoke on the server. Examples: `list_files`, `get_commits`, `send_message`. Tools are the primary way LLMs interact with external systems -- they represent the verbs of the MCP vocabulary.

7.2 Resources (Static)

Structured data the AI can read. Examples: database rows, file contents, API responses. Resources represent the nouns -- the data that tools operate on or that the LLM needs for context.

7.3 Prompts

Pre-built prompt templates that the server ships alongside its tools. Instead of users writing a detailed review prompt from scratch, the server provides an expert-crafted template -- and every client benefits automatically. This ensures reliability and consistent output quality across all AI clients.



Prompt Primitive Example: GitHub Code Review

A GitHub MCP Server exposes a 'review-pull-request' prompt. When a user says 'review PR #42', the Host

fetches the template from the server, fills in the arguments (pr_number=42, focus='security'), and injects the rendered prompt into the LLM context. The user gets an expert-quality review without writing a single line of prompt engineering.

Without Prompt Primitive	With Prompt Primitive
User writes detailed review prompt	Host fetches template automatically
Prompt quality varies per client	Server ships expert prompts
Every client re-implements logic	Update once, all clients benefit

8. Standard Operations

Each primitive type has standard operations that the client can call via JSON-RPC 2.0:

Primitive	Operation	Description
Tools	tools/list	Discover available tools
Tools	tools/call	Invoke a tool with arguments
Resources	resources/list	List available resources
Resources	resources/read	Read a specific resource
Resources	resources/subscribe	Watch for resource changes
Prompts	prompts/list	List available prompt templates
Prompts	prompts/get	Retrieve a rendered prompt

9. Data Layer: Why JSON-RPC 2.0, Not REST

All MCP messages use JSON-RPC 2.0 -- a lightweight RPC standard. Every interaction is one of three message types: Request (client to server, expects response), Response (server to client, result or error), or Notification (either direction, fire-and-forget).

REST might seem like the obvious choice, but MCP's design makes JSON-RPC 2.0 the better fit for three reasons:

Reason 1: Procedure Calls, Not CRUD

REST is built around resources -- you GET, POST, PUT, or DELETE a URL. MCP operations are actions: tools/call, prompts/get, sampling/createMessage. Forcing these into REST produces awkward patterns like POST /tools/{name}/call -- which is just RPC in a REST costume.

Reason 2: Transport Agnosticism

REST is semantically tied to HTTP. JSON-RPC 2.0 is a message envelope -- the same format works over stdio, WebSocket, or HTTP POST. This is essential for local MCP servers that run as a subprocess and communicate over standard input/output, with no HTTP layer at all.

Reason 3: Native Notifications

JSON-RPC 2.0 has built-in support for fire-and-forget notifications. MCP uses these for progress updates during long-running tool calls, log streaming, and resource change events. REST has no native equivalent -- you'd need to bolt on SSE or WebSockets as a separate concern.

Aspect	REST	JSON-RPC 2.0
Mental model	Nouns (resources)	Verbs (procedures)
Transport	HTTP only	Any: stdio, WS, HTTP
Bidirectional	No	Yes (notifications)
Batching	Not standard	Built-in
MCP fit	Awkward	Natural

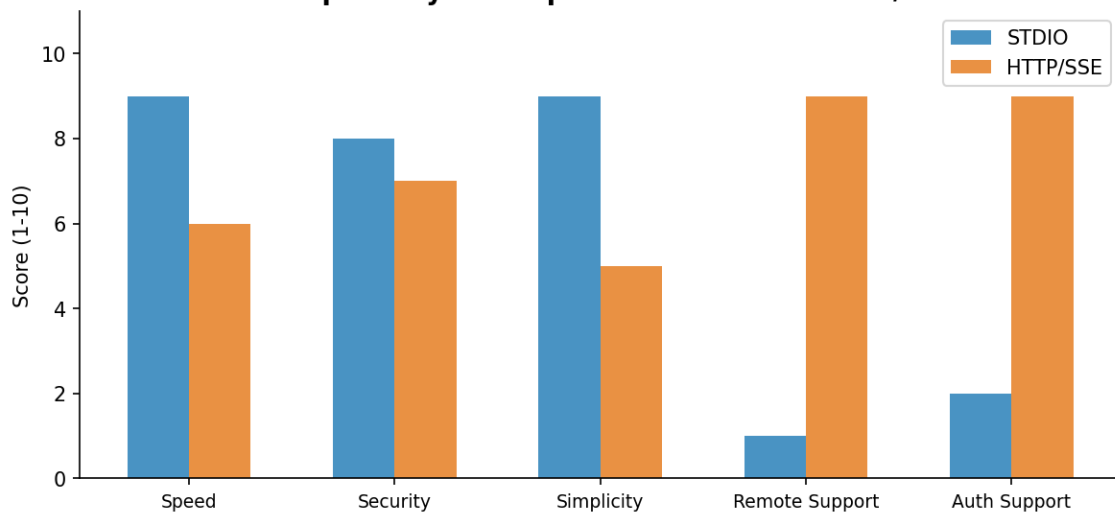
10. Transport Layer: Local vs. Remote

The transport layer is how JSON-RPC messages physically move between client and server. MCP supports two transports, each suited to different deployment scenarios:

Transport	Server Type	Best For
STDIO	Local (same machine)	Fast, secure, simple -- CLI tools & plugins
HTTP/SSE	Local or Remote	Remote calls, HTTP-compatible auth & encryption

A local file-system MCP server runs as a child process and communicates over stdin/stdout. A GitHub MCP server runs remotely and receives calls over HTTP. SSE (Server-Sent Events) enables streaming -- sending data incrementally over a single open connection.

Transport Layer Comparison: STDIO vs HTTP/SSE



11. MCP Session Lifecycle

Every MCP session passes through three phases in order: Initialization, Operation, and Shutdown. This guarantees that both sides know each other's capabilities before any work begins, and that every session closes cleanly.

Phase 1: Initialization

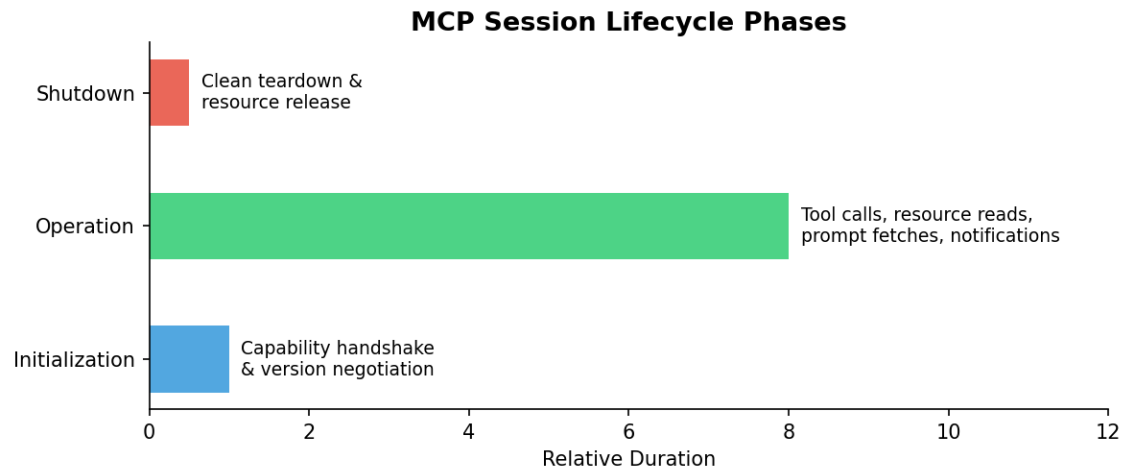
A three-step handshake establishes a shared understanding of what both sides can do. The client sends its protocol version and capabilities, the server responds with its own, and the client confirms with a notifications/initialized message. If protocol versions are incompatible, the server rejects the connection immediately. No tool calls or resource reads can happen until initialization completes.

Phase 2: Operation

Once initialized, the client calls tools, reads resources, and fetches prompts. The server can push notifications at any time -- progress updates, log messages, resource change events -- without waiting for a request. This is where all the real work happens.

Phase 3: Shutdown

The client always initiates shutdown. It waits for in-flight requests to finish, sends a shutdown request, then sends an exit notification. If the transport drops unexpectedly (crash, network loss), the server treats it as an implicit shutdown and releases all session resources.



12. MCP Libraries and FastAPI Compatibility

Currently there are two main libraries for implementing MCP servers and clients:

Library	Source	Version	Notes
mcp	Anthropic (official SDK)	Includes FastMCP 1.0	pip install mcp
fastmcp	Community fork	v2.0	pip install fastmcp

Both libraries share the same interfaces and are largely code-compatible. Notably, FastMCP is designed to be compatible with FastAPI by design. If a company has an existing FastAPI server, it can be wrapped as an MCP server with minimal effort:

```
from main import app # existing FastAPI application
from fastmcp import FastMCP

mcp = FastMCP.from_fastapi(app=app, name="mymcp")
# Now the existing app is MCP-compatible!
```

13. Demo: Building MCP Servers

Demo 1: Simple Server (add & greet)

The simplest MCP server exposes basic tools using the `@mcp.tool()` decorator. Each function becomes a callable tool that any MCP client can discover and invoke:

```
from fastmcp import FastMCP

mcp = FastMCP("Demo Server")
```

```

@mcp.tool()
def add(a: int, b: int) -> int:
    """Add two numbers together."""
    return a + b

@mcp.tool()
def greet(name: str) -> str:
    """Return a greeting for the given name."""
    return f"Hello, {name}!"

if __name__ == "__main__":
    mcp.run()

```

Demo 2: Expense Tracker with SQLite

A more realistic example: an expense tracker MCP server backed by SQLite. It exposes full CRUD operations as MCP tools, allowing any AI client to manage expenses through natural language:

Tool	Parameters	Description
add_expense	description, amount	Add a new expense
get_expenses	(none)	List all expenses
update_expense	id, description, amount	Update an expense
delete_expense	expense_id	Delete an expense by ID
get_total_expenses	(none)	Sum of all expenses
get_expenses_by_date	start_date, end_date	Filter by date range

The server initializes a SQLite database on startup and each tool function handles its own database connection. This pattern -- wrapping an existing data store behind MCP tool decorators -- is the most common way to build production MCP servers.

Conclusion: Key Takeaways

MCP's value is not any single feature -- it's the whole system working together:

- Primitives define what servers can offer: tools, resources, and prompts.
- JSON-RPC 2.0 defines the common language for all messages.
- The transport layer handles local (STDIO) and remote (HTTP/SSE) delivery.
- The lifecycle guarantees clean session management end-to-end.
- The 1:1 connection model keeps servers isolated and independently deployable.

The result is an ecosystem where any AI client can talk to any MCP server, out of the box, with no custom glue code. Adding a new service to the ecosystem instantly makes it available to every AI that speaks MCP.

"The $N \times M$ integration tax becomes $N + M$. That's the promise of MCP."